Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit,
Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. We propose the Transformer, a model architecture eschewing recurrence and relying entirely on attention mechanisms to draw global dependencies between input and output.

1. Introduction

Recurrent neural networks, particularly LSTM and gated recurrent architectures, have been firmly established as state of the art approaches in sequence modeling. Multi-head self-attention allows the model to jointly attend to information from different representation subspaces at different positions.

2. Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure. The encoder maps an input sequence of symbol representations (x1, ..., xn) to a sequence of continuous representations z = (z1, ..., zn).

2.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of N = 6 identical layers.
Each layer has two sub-layers: multi-head self-attention and position-wise feed-forward.
We employ residual connections around each of the two sub-layers, followed by layer normalization: LayerNorm(x + SubLayer(x)). All sub-layers produce outputs of dimension d_model = 512.

3. Attention Mechanisms

An attention function maps a query and a set of key-value pairs to an output,
where the query, keys, values, and output are all vectors.

3.1 Scaled Dot-Product Attention

We call our particular attention Scaled Dot-Product Attention.
The input consists of queries and keys of dimension dk, and values of dimension dv.

Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V

We compute the dot products of the query with all keys, divide each by sqrt(d_k),
and apply a softmax function to obtain the weights on the values.

3.2 Multi-Head Attention

Multi-Head Attention projects queries, keys, and values h = 8 times with different linear projections.

3.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

1. In encoder-decoder attention layers, queries come from previous decoder layer, and memory keys and values come from the output of the encoder.
2. The encoder contains self-attention layers where all keys, values and queries come from the previous layer in the encoder.
3. Self-attention layers in the decoder allow each position to attend to all positions up to and including that position.

3.4 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder
contains a fully connected feed-forward network, applied to each position separately and identically.
FFN(x) = max(0, xW1 + b1) W2 + b2

3.5 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model.

3.6 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position.

PE(pos, 2i) = sin(pos / 10000^(2i / d_model))

PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))

4. Why Self-Attention

In this section we compare various aspects of self-attention layers to recurrent and convolutional layers.

1. Total computational complexity per layer.
2. Amount of computation that can be parallelized, measured by minimum sequential operations.
3. Path length between long-range dependencies in the network.

4.1 Computational Complexity Comparison

Layer Type	Complexity per Layer	Sequential Ops	Maximum Path Length
Self-Attention	O(n^2 * d)	O(1)	O(1)
Recurrent	O(n * d^2)	O(n)	O(n)
Convolutional	O(k * n * d^2)	O(1)	O(log_k(n))
Self-Attention (r)	O(r * n * d)	O(1)	O(n/r)

5. Training Details

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence p
Sentences were encoded using byte-pair encoding, which has a shared vocabulary of about 37000 tokens

5.2 Hardware and Schedule
We trained our models on one machine with 8 NVIDIA P100 GPUs.
For base models using hyperparameters d_model=512, h=8, d_ff=2048, each step took 0.4 seconds.
Total training time: 100,000 steps (12 hours).
For big models, step time was 1.0 seconds for 300,000 steps (18 hours).

5.3 Optimizer Settings
We used the Adam optimizer with beta1 = 0.9, beta2 = 0.98 and epsilon = 10^-9.
The learning rate was varied over training according to:
lrate = d_model^(-0.5) * min(step_num^(-0.5), step_num * warmup_steps^(-1.5))
where warmup_steps = 4000.

5.4 Regularization
We employ three types of regularization during training:
1. Residual Dropout: We apply dropout to output of each sub-layer before it is added to sub-layer input.
2. Label Smoothing: During training, we employed label smoothing of epsilon_ls = 0.1.

6. Results & Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model achieves 28.4 BLEU.
On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.8.
Outperforming all of the previously published single models and ensembles at a fraction of training cost.

5.1 Hardware and Schedule (Detailed Training Log)
We trained our models on one machine with 8 NVIDIA P100 GPUs.
For the base models using the hyperparameters described throughout the paper, each training step took a
We trained the base models for a total of 100,000 steps or 12 hours.
For our big models, step time was 1.0 seconds. The big models were trained for 300,000 steps (18 hours).

Table 2: Comparison with state-of-the-art translation models:

Model	BLEU (En-De)	BLEU (En-Fr)	Training Cost (FLOPs)
Transformer (base)	27.3	38.1	3.3 * 10^18
Transformer (big)	28.4	41.8	2.3 * 10^19

7. Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attent
replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-head self-a
For translation tasks, the Transformer can be trained significantly faster than architectures based on recurr
We are excited about the future of self-attention models and plan to apply them to other tasks.